

0) Visão do MVP (o que entra e o que fica fora)

Objetivo do MVP: provar, com software funcional, que uma “camada de governança algorítmica” consegue **avaliar e decidir** (allow/review/block) respostas de um LLM usando métricas quantificáveis (δ , ψ , ρ , ω simplificada) e gerar **logs auditáveis + dashboard**.

Inclui (agora):

- **Kai Core** (roteamento Fast/Deep Path)
- **MIR v0:** δ (coerência) e ψ (estabilidade) simplificados; ρ (bias) básico; ω (auditabilidade) simples
- **CATE v0:** logging estruturado + explicação curta da decisão
- **Dashboard v0:** métricas em tempo real e replay (shadow mode)
- **MACT v0** (simulado): thresholds por domínio + ajuste manual

Fica para depois (não trava o MVP):

- MACT com aprendizado (contextual bandit)
 - Nested Evaluation com CoT explícito
 - ω completo (trace causal + contrafactuais)
 - Benchmark científico público (Cluster S completo)
-

1) Arquitetura de referência (MVP)

Stack sugerido (100% viável no Base44):

- **Gateway/API:** FastAPI (Python) ou Node/Express
- **LLM Client:** OpenAI/Anthropic/Mistral SDK
- **Métricas:** Python (Transformers + sentence-transformers)
- **Fila opcional:** Redis (para deep path assíncrono)
- **Banco:** PostgreSQL (prod) ou SQLite (dev)
- **Vector store:** FAISS (local) para embeddings
- **Dashboard:** Streamlit ou Gradio
- **Observability:** Prometheus + Grafana (opcional no MVP)
- **Infra:** Docker + docker-compose

Fluxo (Fast/Deep Path):

1. POST /v1/evaluate recebe input → **Fast Path** roda heurísticas baratas (δ_{nli} rápido + detecção de risco via palavras-chave + entropia/logprobs quando disponível).

2. Se **risco baixo**, retorna decisão “allow” com **latência** $\leq 80\text{ms}$.
 3. Se **risco médio/alto**, dispara **Deep Path** (em paralelo ou síncrono): δ completo + ψ por amostragem reduzida + ρ básico + ω_{audit} . Retorna “allow/review/block” com **latência** $\leq 150\text{--}200\text{ms}$.
 4. **CATE** registra trace completo (métricas, thresholds, decisão, rationale) no **Ledger** (tabela APP_LOGS + POP).
 5. **Dashboard** mostra as decisões e permite **replay** (shadow mode).
-

2) Estrutura de pastas (sugestão)

```
terkai-mvp/  
  api/  
    main.py           # FastAPI/Express endpoints  
    schemas.py        # Pydantic/Types para requests/responses  
    config.py         # env, keys, domains  
  core/  
    router.py         # Fast/Deep path decision  
    llm_client.py     # wrappers p/ OpenAI/Anthropic/Mistral  
    thresholds.py     # thresholds por domínio (MACT v0)  
  metrics/  
    delta.py         #  $\delta$  (NLI + factual + negação simples)  
    psi.py           #  $\psi$  (logprobs ou 3 amostras)  
    rho.py           #  $\rho$  básico (detector de estereótipos + keywords)  
    omega.py         #  $\omega_{\text{audit}}$  (reprodutibilidade + POP completeness)  
    utils.py         # tokenização, embeddings, normalizações  
  cate/  
    logger.py        # logging estruturado (JSON)  
    rationale.py     # geração de explicação curta  
  storage/  
    db.py            # conexão com Postgres/SQLite  
    models.sql       # criação de tabelas  
  dashboard/  
    app.py           # Streamlit/Gradio  
  tests/  
    unit/           # testes unitários  
    e2e/            # testes ponta-a-ponta com dados sintéticos  
  docker/  
    Dockerfile  
    docker-compose.yml  
  README.md
```

3) Esquemas de dados (DB e POP)

Tabela requests (entrada/saída):

- id (uuid, pk)
- timestamp
- tenant_id / domain (ex.: finance/health/general)
- input_text (text)

- `model_name`, `model_version`
- `fast_path_used` (bool), `deep_path_used` (bool)
- `llm_output` (text)
- `decision` (allow|review|block)
- `ephi_score` (float) — se calculado
- `latency_ms` (int)

Tabela `metrics` (por request):

- `request_id` (fk)
- `delta`, `psi`, `rho`, `omega_audit` (floats 0–1)
- `delta_components` (json: nli, factual, negation)
- `psi_method` (enum: logprobs|semantic_entropy|multi_samples)
- `rho_components` (json: stereotypes, keywords, balance)
- `pop_completeness` (0–1)
- `weights_used` (json: α , β , γ , η)
- `thresholds_used` (json)

Tabela `pop` (Policy & Operations Pack — auditável):

- `request_id` (fk)
- `policy_version` (string)
- `prompt_template_version` (string)
- `rules_applied` (json)
- `explanations` (text)
- `seed` (int|null)
- `replay_hash` (sha256 de input+config)

4) Especificação da API (MVP)

POST `/v1/evaluate`

Request (JSON):

```
{
  "tenant_id": "acme",
  "domain": "finance",
  "input": "Texto do usuário",
  "params": {
```

```
    "temperature": 0.4,  
    "max_tokens": 300  
  }  
}
```

Response (JSON):

```
{  
  "decision": "allow",  
  "output": "Resposta do LLM (ou alternativa segura)",  
  "metrics": {  
    "delta": 0.86,  
    "psi": 0.78,  
    "rho": 0.91,  
    "omega_audit": 0.62,  
    "ephi": 0.83  
  },  
  "rationale": "Sem contradições; baixo risco; logging completo",  
  "trace_id": "uuid",  
  "latency_ms": 94  
}
```

5) Implementação das métricas (v0 pragmático)

5.1 δ (Coerência) — prioridade alta

- **δ_{nli} (70%):** modelo NLI leve (DeBERTa-base) para contradição vs contexto (ou só contra o input quando não houver contexto).
- **δ_{factual} (20%):** procura no próprio input/contexto frases que suportem os claims da resposta (BM25/embeddings + cross-encoder leve).
- **δ_{negation} (10%):** heurística de negação explícita (“não”, “nunca”, “impossível”) cruzada com claims do contexto.

Aceite MVP: δ correlaciona com avaliação humana (amostra 100–200) $\rho \geq 0,6$.

5.2 ψ (Estabilidade) — trade-off custo/latência

- **Método A (preferido quando disponível):** usar **logprobs**/entropia média de tokens (1 chamada).
- **Fallback (geral):** gerar **3 amostras** (temp=0.7) e medir dispersão semântica (embeddings + distância ao centróide); **só no Deep Path**.

Aceite MVP: ψ baixo prediz casos instáveis (validação pequena: 150–200 prompts).

5.3 ρ (Imparcialidade) — básico e conservador

- Detector de **atributos protegidos** (NER + regex).
- **Lista de estereótipos**/termos proibidos versionada.

- Classificador leve (zero-shot) para marcar suspeitas em **gênero/raça/idade/religião**.
- **Penalizar** ρ quando houver match explícito; manter conservador (melhor false positive do que false negative no MVP).

Aceite MVP: ρ sinaliza corretamente >70% dos casos sintéticos com vieses óbvios.

5.4 ω (Auditabilidade/Reprodutibilidade) — simples

- ω_{audit} = média ponderada de:
 - seed fixo? (bool)
 - policy_version presente? (bool)
 - model_version presente? (bool)
 - **POP completeness** (% campos preenchidos)
- **Meta:** ω não “julga moralmente”, apenas qualidade de replay/auditoria.

Aceite MVP: $\omega \geq 0,7$ quando todos campos estão versionados + seed fixo.

6) $E\phi$ (Score composto) + Decisão

Pesos fixos por domínio (MVP):

- finance: $\alpha=0.35$ (δ), $\beta=0.30$ (ψ), $\gamma=0.20$ (ρ), $\eta=0.15$ (ω)
- health: $\alpha=0.40$, $\beta=0.35$, $\gamma=0.15$, $\eta=0.10$
- general: $\alpha=\beta=\gamma=\eta=0.25$

Decisão:

- $e\phi \geq \text{safe_threshold} \rightarrow$ **allow**
- $\text{review_threshold} \leq e\phi < \text{safe_threshold} \rightarrow$ **review** (retorna output + “banner de revisão” + rationale)
- $e\phi < \text{review_threshold} \rightarrow$ **block** (gera **alternative safe output** com explicação neutra)

MVP thresholds por domínio, calibrados manualmente após 100–200 casos anotados.

7) CATE (logging + rationale)

- Armazena JSON completo por decisão (métricas, pesos, thresholds, tempo, seed, versões).
- Gera **rationale curto** (duas frases) explicando a decisão com base nas métricas (regra determinística, nada “filosófico”).

- Mantém **POP** com:
 - `policy_version, model_version, prompt_template_version`
 - `rules_applied, weights_used, thresholds_used`
 - `replay_hash` (sha256: input + config)
-

8) Dashboard (Streamlit/Gradio)

Páginas:

1. **Overview:** throughput, p50/p95 latência, % allow/review/block, distribuição de $\delta/\psi/\rho/\omega$, top alerts.
2. **Requests:** tabela com filtros (domínio, decisão, score), **replay** de decisão, download do POP.
3. **Policies:** editar thresholds por domínio (arquivo YAML versionado).
4. **Sandbox:** caixa de texto para testar prompts e ver métricas ao vivo.

Aceite MVP: p95 latência < 150–200ms (com 20% Deep Path), p50 < 80ms.

9) Testes e validação (mínimo para credibilidade)

- **Unit tests** (metrics, roteador, logger) — 70%+ de cobertura nos módulos críticos.
- **E2E** com 150–300 prompts sintéticos (curados por você):
 - 50 “seguros”
 - 50 “ambíguos”
 - 50 “problemáticos” (viés, contradição, instabilidade)
- **Validação humana:** você classifica manualmente 150 casos → medir:
 - Correlação δ vs sua avaliação de coerência
 - ψ baixo nos casos instáveis
 - ρ sinaliza os vieses óbvios (sem muito falso positivo)
 - **Confusion matrix** das decisões (allow/review/block) vs rótulo humano

Entrega de validação (PDF curto, 3–5 páginas):

- Metodologia + tabelas (precisão/recall/F1 por classe)
 - 8–10 exemplos anotados (screenshots do dashboard)
 - Limitações conhecidas (ex.: ψ custoso quando sem logprobs)
-

10) Cronograma sugerido (4–6 semanas, solo)

Semana 1 – Setup & esqueleto

- Repositório + docker-compose
- FastAPI + endpoint `/v1/evaluate`
- LLM wrapper (OpenAI/Anthropic, alternável por env)
- DB (SQLite no dev)
- Esqueleto do dashboard

Semana 2 – Métricas δ e ψ

- δ_{nli} + δ_{negation} + δ_{factual} (v0)
- ψ_{logprobs} (e fallback 3 amostras no Deep Path)
- Roteador Fast/Deep Path (regra simples)
- Logging (CATE v0) + POP v0

Semana 3 – ρ e ω simples + decisão $E\phi$

- ρ básico (atributos + keywords)
- ω_{audit} (seed/versionamento/POP completeness)
- $E\phi$ com pesos por domínio + thresholds fixos
- “Safe alternative output” (template conservador)

Semana 4 – Dashboard + testes

- Streamlit: overview + requests + sandbox
- E2E tests com 150–300 prompts
- Ajuste de thresholds baseado nos seus rótulos
- Documento de validação (3–5 páginas)

(Opcional) Semana 5–6 – Endurecer

- Redis fila para Deep Path assíncrono
- Otimizações (cache ψ por input normalizado)
- Docker pronto para demo cloud
- Vídeo demo (2–3 min) clicável

11) O que simular (para “demonstrar conceito”)

- **MACT v0 (simulado)**: painel para “ajustar pesos/thresholds” por domínio e registrar um *changelog*. Deixe claro que é **manual** no MVP.

- **Nested Eval (proxy):** se o provedor não expõe CoT, registre **perplexidade** / **entropia** por token como “IRRS proxy”.
 - **ρ :** use dicionário de termos e 4–5 frases *seed* para cada viés (básico, mas demonstrável).
-

12) Entregáveis do MVP (tudo que você consegue com Base44)

1. **Repositório** (privado) com:
 - API running (FastAPI/Express)
 - Métricas v0 (δ , ψ , ρ , ω)
 - Decisor $E\phi$
 - CATE/POP
 - Docker
 2. **Dashboard funcional** (Streamlit/Gradio)
 3. **Documento técnico do MVP** (10–15 páginas):
 - Arquitetura, métricas (definições e limites), endpoints
 - Resultados de validação pequena
 - Roadmap claro para MACT e Cluster S
 4. **Vídeo demo** (2–3 minutos)
 5. **Draft de patente provisória** (claims do método de decisão composto + POP + roteamento Fast/Deep + explicabilidade determinística)
-

13) Riscos e mitigação (MVP)

- **ψ custoso** → rodar apenas no Deep Path; cache; usar logprobs se possível.
 - **Falsos positivos em ρ** → mantenha “review” (em vez de “block”) para casos ambíguos.
 - **Latência** → cuidado com cross-encoders; use heurísticas e reduza K quando possível.
 - **Generalização** → declare **escopo do MVP** (domínios testados, linguagem PT/EN).
-

14) Próximos passos (pós-MVP)

- MACT v1 (feedback humano → ajuste de thresholds)
- δ/ψ robustos com **benchmarks públicos** e paper curto (arXiv)
- ω com SHAP/LIME (explicabilidade real)
- Piloto controlado (shadow mode) com 1 parceiro

Resumo executivo (1 parágrafo para você usar)

O MVP do TER KAI será uma API com roteamento Fast/Deep Path que avalia respostas de LLM com métricas δ (coerência), ψ (estabilidade), ρ (imparcialidade) e ω (auditabilidade) simplificadas, gera um score composto $E\phi$ e decide allow/review/block com logging auditável (CATE/POP) e dashboard ao vivo. Em 4–6 semanas é possível entregar um protótipo funcional e medido (150–300 casos), suficiente para demonstração a parceiros e para registrar patente provisória, deixando aprendizado adaptativo (MACT) e validação científica ampla para a próxima fase.

Mensagem 1 — API, Roteador e Fluxo de Decisão

1) Especificação de Endpoints (MVP)

POST /v1/evaluate

- **Entrada:** tenant/domain, input, parâmetros de geração
- **Saída:** decisão (allow/review/block), output, métricas, rationale, trace_id, latência

Contrato (pseudocódigo JSON):

```
// Request
{
  "tenant_id": "acme",
  "domain": "finance",
  "input": "texto do usuário",
  "params": {
    "temperature": 0.4,
    "max_tokens": 300,
    "model": "gpt-4o-mini"
  }
}

// Response
{
  "decision": "allow",
  "output": "resposta do LLM",
  "metrics": {
    "delta": 0.86,
    "psi": 0.78,
    "rho": 0.91,
    "omega_audit": 0.62,
    "ephi": 0.83
  },
  "rationale": "Sem contradições com o contexto; baixa incerteza; POP completo.",
  "trace_id": "uuid-...-...",
  "latency_ms": 94
}
```

```
}
```

GET /v1/requests/{trace_id}

- Retorna o **POP** + métricas + payloads para auditoria/replay.

POST /v1/policies/apply

- Atualiza **thresholds** e **pesos** por domínio (MACT v0 manual).
- Corpo: { "domain": "finance", "weights": {...}, "thresholds": {...}, "changelog_note": "ajuste após calibração inicial" }.

GET /v1/health

- Para readiness/liveness.

2) Roteador Fast/Deep Path (pseudocódigo Python-like)

```
# core/router.py
from time import perf_counter
from core.llm_client import generate_llm
from metrics.delta import delta_fast, delta_full
from metrics.psi import psi_fast_or_skip
from metrics.rho import rho_basic
from metrics.omega import omega_audit
from core.thresholds import get_domain_weights, get_domain_thresholds
from cate.logger import log_trace
from cate.rationale import build_rationale

def evaluate_request(req):
    t0 = perf_counter()

    tenant = req.tenant_id
    domain = req.domain or "general"
    input_text = req.input
    params = req.params or {}

    # --- 1) Geração do output base (stream off no MVP)
    llm_out, model_info = generate_llm(input_text, params)

    # --- 2) FAST PATH: sinais baratos/rápidos
    #  $\delta_{fast}$ : NLI leve e negações explícitas;  $\psi_{fast}$ : logprobs se disponível;  $\rho$ :
    # keywords/NER simples
    delta_fp = delta_fast(input_text, llm_out)
    psi_fp = psi_fast_or_skip(input_text, llm_out, model_info)
    rho_fp = rho_basic(llm_out)
    omega_fp = omega_audit(pop=None, model_info=model_info,
seed=params.get("seed"))

    # --- 3) Score composto provisório (E $\phi$  preliminar)
    weights = get_domain_weights(domain) # ex: {alpha, beta, gamma, eta}
    thresholds = get_domain_thresholds(domain) # ex: {safe, review}
    ephi_fast = (
        weights.alpha * delta_fp +
        weights.beta * psi_fp +
        weights.gamma * rho_fp +
        weights.eta * omega_fp
```

```

)

# --- 4) Decisão Fast Path
if ephi_fast >= thresholds.safe:
    decision = "allow"
    metrics = {"delta": delta_fp, "psi": psi_fp, "rho": rho_fp,
"omega_audit": omega_fp, "ephi": ephi_fast}
    rationale = build_rationale(metrics, decision, fast_path=True)
    latency_ms = int((perf_counter() - t0) * 1000)

    trace_id = log_trace(
        tenant=tenant, domain=domain, input_text=input_text,
output_text=llm_out,
        model_info=model_info, metrics=metrics, weights=weights,
thresholds=thresholds,
        decision=decision, latency_ms=latency_ms, path="fast"
    )
    return decision, llm_out, metrics, rationale, trace_id, latency_ms

# --- 5) DEEP PATH: métricas completas (custo/latência maior)
delta_dp = delta_full(input_text, llm_out) # NLI+factual+negação composta
psi_dp = psi_fast_or_skip(input_text, llm_out, model_info, deep=True) #
logprobs; se não, 3 amostras
rho_dp = rho_basic(llm_out) # mantém básico no MVP
omega_dp = omega_audit(pop=None, model_info=model_info,
seed=params.get("seed"))

ephi_deep = (
    weights.alpha * delta_dp +
    weights.beta * psi_dp +
    weights.gamma * rho_dp +
    weights.eta * omega_dp
)

# --- 6) Decisão final
if ephi_deep >= thresholds.safe:
    decision = "allow"
elif ephi_deep >= thresholds.review:
    decision = "review"
else:
    decision = "block"

# --- 7) Se bloquear, gerar saída segura alternativa
safe_output = None
if decision == "block":
    safe_output = (
        "Não posso responder diretamente a essa solicitação. "
        "Posso fornecer informações de alto nível ou orientações seguras."
    )

# --- 8) Rationale + logging POP/CATE
metrics = {"delta": delta_dp, "psi": psi_dp, "rho": rho_dp, "omega_audit":
omega_dp, "ephi": ephi_deep}
rationale = build_rationale(metrics, decision, fast_path=False)
latency_ms = int((perf_counter() - t0) * 1000)

output_final = llm_out if decision != "block" else safe_output
trace_id = log_trace(
    tenant=tenant, domain=domain, input_text=input_text,
output_text=output_final,
    model_info=model_info, metrics=metrics, weights=weights,
thresholds=thresholds,
    decision=decision, latency_ms=latency_ms, path="deep"
)

```

```
return decision, output_final, metrics, rationale, trace_id, latency_ms
```

Observações práticas:

- **Fast Path** cobre ~70–85% dos casos com latência baixa.
 - **Deep Path** só dispara quando $E\phi_{\text{fast}} < \text{safe}$ (casos ambíguos/riscados).
 - ψ no Deep Path usa **logprobs** (1 chamada) quando o provedor expõe; fallback **3 amostras** (paralelizadas) para custo controlado.
-

3) Wrapper do LLM (troca de provedor por env)

```
# core/llm_client.py
import os

def generate_llm(prompt, params):
    provider = os.getenv("LLM_PROVIDER", "openai")
    model = params.get("model") or os.getenv("LLM_MODEL", "gpt-4o-mini")
    temperature = params.get("temperature", 0.2)
    max_tokens = params.get("max_tokens", 300)

    if provider == "openai":
        # chamada OpenAI; quando possível, pedir logprobs
        out_text, meta = openai_generate(prompt, model, temperature, max_tokens,
want_logprobs=True)
        return out_text, {"provider": "openai", "model": model,
"supports_logprobs": meta["supports_logprobs"]}
    elif provider == "anthropic":
        out_text, meta = anthropic_generate(prompt, model, temperature,
max_tokens)
        return out_text, {"provider": "anthropic", "model": model,
"supports_logprobs": meta.get("supports_logprobs", False)}
    else:
        out_text, meta = generic_llm_generate(prompt, model, temperature,
max_tokens)
        return out_text, {"provider": provider, "model": model,
"supports_logprobs": meta.get("supports_logprobs", False)}
```

4) Decisão $E\phi$ + Pesos/Thresholds (MACT v0 manual)

```
# core/thresholds.py
DOMAIN_WEIGHTS = {
    "finance": {"alpha": 0.35, "beta": 0.30, "gamma": 0.20, "eta": 0.15},
    "health": {"alpha": 0.40, "beta": 0.35, "gamma": 0.15, "eta": 0.10},
    "general": {"alpha": 0.25, "beta": 0.25, "gamma": 0.25, "eta": 0.25}
}

DOMAIN_THRESHOLDS = {
    "finance": {"safe": 0.85, "review": 0.70},
    "health": {"safe": 0.87, "review": 0.72},
    "general": {"safe": 0.75, "review": 0.60}
}

def get_domain_weights(domain):
    return SimpleNamespace(**DOMAIN_WEIGHTS.get(domain,
DOMAIN_WEIGHTS["general"]))
```

```
def get_domain_thresholds(domain):
    return SimpleNamespace(**DOMAIN_THRESHOLDS.get(domain,
DOMAIN_THRESHOLDS["general"]))
```

5) Rationale determinístico (CATE)

```
# cate/rationale.py
def build_rationale(metrics, decision, fast_path: bool) -> str:
    d = metrics["delta"]; p = metrics["psi"]; r = metrics["rho"]; o =
metrics["omega_audit"]
    if decision == "allow":
        return (
            f"Decisão: allow. Coerência alta ( $\delta$ ={d:.2f}), estabilidade adequada
( $\psi$ ={p:.2f}), "
            f"sem indícios de viés ( $\rho$ ={r:.2f}) e auditabilidade aceitável
( $\omega$ ={o:.2f}). "
            f"{'Fast' if fast_path else 'Deep'} Path."
        )
    if decision == "review":
        return (
            f"Decisão: review. Métricas indicam risco moderado:  $\delta$ ={d:.2f},
 $\psi$ ={p:.2f},  $\rho$ ={r:.2f},  $\omega$ ={o:.2f}. "
            "Recomenda-se revisão humana."
        )
    return (
        f"Decisão: block. Métricas insuficientes ( $\delta$ ={d:.2f},  $\psi$ ={p:.2f},  $\rho$ ={r:.2f},
 $\omega$ ={o:.2f}). "
        "Foi fornecida resposta alternativa segura."
    )
```

6) Logging POP/CATE (estrutura mínima)

```
# cate/logger.py
import uuid
from storage.db import insert_request, insert_metrics, insert_pop

def log_trace(tenant, domain, input_text, output_text, model_info, metrics,
weights, thresholds, decision, latency_ms, path):
    trace_id = str(uuid.uuid4())

    insert_request({
        "id": trace_id,
        "tenant": tenant,
        "domain": domain,
        "input": input_text,
        "output": output_text,
        "model_name": model_info["provider"] + ":" + model_info["model"],
        "path": path,
        "decision": decision,
        "ephi": metrics["ephi"],
        "latency_ms": latency_ms
    })

    insert_metrics(trace_id, metrics, weights, thresholds)
    insert_pop(trace_id, {
        "policy_version": "v0.1",
```

```
    "prompt_template_version": "v0.1",
    "rules_applied": ["fast_path" if path=="fast" else "deep_path"],
    "model_version": model_info["model"],
    "seed": None, # se houver
    "replay_hash": compute_replay_hash(input_text, model_info, weights,
thresholds)
    })

    return trace_id
```

Se estiver ok, na **Mensagem 2** trago os **pseudocódigos das métricas** (δ , ψ , ρ , ω) no nível de detalhe certo para você (ou a Base44) implementar sem dúvidas, incluindo opções de fallback, parâmetros tuning e custos/latência estimados por componente.

Perfeito — aqui vai a **Mensagem 2** com os **pseudocódigos das métricas (MVP)**, incluindo variantes Fast/Deep, parâmetros de tuning, dependências e estimativas realistas de latência/custo.

Métricas do MVP (δ , ψ , ρ , ω) — Pseudocódigo e Operação

Convenções

- Todas as métricas retornam valor em **[0, 1]** (maior = melhor).
 - “Fast Path” = heurísticas baratas; “Deep Path” = verificações completas.
 - Estimativas de latência p/ CPU; on-GPU pode reduzir em 30–60%.
 - Onde houver *provider support* (ex. logprobs), habilitamos; senão, usamos fallback.
-

1) δ — Coerência (consistência lógica/factual com o contexto)

Objetivo

Reduzir contradições explícitas e incoerências óbvias entre **contexto** e **resposta** sem abrir web search (MVP).

Dependências

- NLI leve (ex.: deberta-v3-base-mnli ou miniLM-mnli ONNX).
- Tokenizador/NER simples.
- Reranker cross-encoder *pequeno* (opcional no Deep Path).

Parâmetros (sugestão inicial)

$W_{NLI}=0.7$, $W_{FACT}=0.2$, $W_{NEG}=0.1$

```
NLI_CONTRAD_PENALTY=1.0, NLI_NEUTRAL=0.5, NLI_ENTAIL=1.0
TOPK_FACT=3
DELTA_FAST_PASS=0.85    # atalho: se  $\delta_{fast} \geq 0.85$ , não faz  $\delta_{full}$ 
```

Fast Path (δ_{fast})

```
def delta_fast(context: str, response: str) -> float:
    # 1) NLI raso
    # Quebrar contexto em sentenças curtas; limitar a N sentenças mais
    # relevantes por embedding cosine
    ctx_sents = select_relevant_sentences(context, response, k=5) # (~1-2ms
    # BM25/embeddings)
    nli_scores = []
    for s in ctx_sents:
        label = nli_light(s, response) # retorna {entailment, neutral,
    # contradiction}
        if label == 'contradiction':
            nli_scores.append(0.0)
        elif label == 'neutral':
            nli_scores.append(0.5)
        else:
            nli_scores.append(1.0)
    nli_score = mean(nli_scores) if nli_scores else 0.7

    # 2) Heurística de negação explícita
    neg_conflicts = detect_negation_conflicts(context, response) # regex +
    # dependências simples
    neg_score = 1.0 - clamp(neg_conflicts / (len(ctx_sents) + 1), 0, 1)

    # 3) (Opcional no fast) factual leve com BM25 + cosine
    factual_score = 0.8 if likely_supported_by_context(response, ctx_sents) else
    # 0.6

    # 4) Combinação
    delta = 0.7 * nli_score + 0.2 * factual_score + 0.1 * neg_score
    return clamp(delta, 0.0, 1.0)
```

Latência típica: 8–20 ms (NLI leve em 3–5 pares).

Observação: Se $\delta_{fast} \geq \text{DELTA_FAST_PASS}$, podemos **pular δ_{full}** .

Deep Path (δ_{full})

```
def delta_full(context: str, response: str) -> float:
    # 1) Seleciona TOPK sentenças mais relacionadas ao(s) claim(s) da resposta
    claims = extract_atomic_claims(response) # regex + simples
    cand_sents = select_relevant_sentences(context, claims, k=8)

    # 2) NLI mais intenso (modelo melhor/maior) em pares (claim x sentença)
    nli_vals = []
    for c in claims or [response]:
        sents = topk_by_similarity(c, cand_sents, k=3)
        for s in sents:
            nli_vals.append(score_nli_medium(s, c)) # (1.0 entail; 0.5
    # neutral; 0.0 contrad)
    nli_score = mean(nli_vals) if nli_vals else 0.7

    # 3) Factual com reranker (cross-encoder pequeno)
    fact_scores = []
    for c in claims:
        sents = topk_by_bm25(cand_sents, c, k=TOPK_FACT)
        reranked = cross_encoder_rerank(c, sents) # pega o melhor
        fact_scores.append(sigmoid(reranked.best_score))
```

```

factual_score = mean(fact_scores) if fact_scores else 0.7

# 4) Negação explícita + contradições numéricas (cheça números/datas)
neg_score = 1.0 - normalized_negation_conflicts(context, response)
num_ok = numeric_consistency_check(context, response) # True/False →
1.0/0.0
numeric_score = 1.0 if num_ok else 0.0

# 5) Combinação (reforça componente numérica no deep)
delta = 0.6 * nli_score + 0.2 * factual_score + 0.1 * neg_score + 0.1 *
numeric_score
return clamp(delta, 0.0, 1.0)

```

Latência típica: 35–80 ms (conforme tamanho do contexto e cross-encoder).

Custo: CPU-only ok; ONNX acelera 2–3×.

2) ψ — Estabilidade (robustez da resposta a variações pequenas da entrada)

Estratégia MVP

- **Preferência:** usar **logprobs** (uma só inferência).
- **Fallback:** amostragem semântica com 3–5 paráfrases (paralelizado).

Parâmetros (sugestão)

```

N_SAMPLES=5 (fallback)
PARAM_TEMPERATURE=0.7
SIM_THRESHOLD=0.85 # clustering
ENTROPY_WARN=0.55 # entropia acima disso indica instabilidade

```

Fast/Deep — escolha automática

```

def psi_fast_or_skip(input_text: str, output_text: str, model_info: dict, deep:
bool=False) -> float:
    # 1) Se provider expõe logprobs → método rápido
    if model_info.get("supports_logprobs", False):
        return psi_logprobs(output_text)

    # 2) Sem logprobs:
    if not deep:
        # No Fast Path retornamos um proxy barato (ex.: comprimento e repetição
→ estabilidade)
        return cheap_stability_proxy(output_text) # 0.65–0.85 típico
    else:
        # Deep Path: semantic entropy com amostragem
        return psi_semantic_entropy(input_text, n=N_SAMPLES)

```

Método com logprobs (1 chamada)

```

def psi_logprobs(output_text_with_logprobs) -> float:
    entropies = []
    for tok in output_text_with_logprobs.tokens:
        probs = softmax(tok.top_logprobs) # top-k prob distrib
        H = -sum(p * log(p + 1e-12) for p in probs) # entropia de Shannon
    return sum(entropies) / len(entropies)

```



```

        entropies.append(H)
    mean_H = mean(entropies)
    max_H = log(len(output_text_with_logprobs.top_logprobs_vocab)) # aprox
    psi = 1.0 - clamp(mean_H / max_H, 0, 1)
    return psi

```

Latência: ~0 ms extra (vem na mesma chamada).

Fallback — “semantic entropy” (n amostras)

```

def psi_semantic_entropy(input_text: str, n: int=5) -> float:
    # 1) Gera n respostas (paralelo; temp=0.7)
    samples = parallel_generate(input_text, n=n, temperature=PARAMETER_TEMPERATURE)

    # 2) Embeddings e clustering por similaridade
    embs = embed_many(samples)
    clusters = cluster_by_cosine(embs, threshold=SIM_THRESHOLD) #
    DBSCAN/HDBSCAN

    # 3) Entropia nos clusters (quanto mais clusters, mais instável)
    counts = [len(c) for c in clusters]
    probs = [c/len(samples) for c in counts]
    H_sem = -sum(p * log(p + 1e-12) for p in probs)
    max_H = log(len(samples))

    psi = 1.0 - clamp(H_sem / max_H, 0, 1)
    return psi

```

Latência: 3–5 chamadas LLM (paralelizadas) → +60–150 ms;

Custo: 3–5× tokens (apenas Deep Path).

3) p — Imparcialidade (sinais de viés explícito/implícito na resposta)

Objetivo

Detectar conteúdo potencialmente enviesado na **resposta específica** (não é avaliação global do modelo) usando sinais simples no MVP.

Dependências

- NER leve de **grupos protegidos** (gênero, raça, religião, idade, etc).
- Lista de *keywords* estereotípicas (por idioma).
- Classificador leve opcional para “estereótipo” (tiny-RoBERTa).

Parâmetros (sugestão)

```

W_STEREO=0.7, W_REP=0.2, W_KEY=0.1
STEREO_THRESHOLD=0.6 # acima disso, provável viés
REP_MIN_DIVERSITY=0.3 # baixa diversidade penaliza (quando aplicável)

```

Implementação (MVP)

```
def rho_basic(response: str) -> float:
    # 1) Detecta atributos protegidos no texto
    groups = detect_protected_attributes(response) # ex.: ["gender:female",
"religion:muslim"]

    # Sem referência a grupos → considerar p alto
    if not groups:
        return 0.95

    # 2) Classificador de estereótipos por grupo (opcional no MVP)
    stereo_probs = []
    for g in groups:
        p = stereotype_classifier_prob(response, group=g) # 0..1 (prob de
estereótipo)
        stereo_probs.append(p)
    stereo_penalty = mean(stereo_probs) if stereo_probs else 0.0
    stereo_score = 1.0 - stereo_penalty

    # 3) Representação balanceada (se há múltiplas entidades/grupos numa mesma
sentença)
    rep_score = measure_representation_balance(response) # 0..1 (alto =
balanceado)

    # 4) Keywords explícitas
    key_penalty = match_bias_keyword_list(response) # 0..1 (1.0 se palavras
problemáticas)
    key_score = 1.0 - key_penalty

    # 5) Combinação
    rho = 0.7 * stereo_score + 0.2 * rep_score + 0.1 * key_score
    return clamp(rho, 0.0, 1.0)
```

Latência: 3–8 ms (NER + listas + classif. leve).

Notas: Em idiomas distintos (PT/EN), manter listas separadas e testes.

4) ω — Auditabilidade/Reprodutibilidade (proxy de “reversibilidade”)

Em vez de “reversibilidade” genérica (muito vaga), o MVP mede **auditabilidade reprodutível**: quão bem conseguimos **reconstituir** a decisão e o contexto técnico.

Componentes (MVP)

- **POP completeness** (campos preenchidos no registro).
- **Determinismo** (seed fixo, temp=0 para ações sensíveis, quando aplicável).
- **Versionamento** (modelo, política, prompt).
- **Explicação gerada (CATE)** disponível.

Parâmetros (sugestão)

W_POP=0.3, W_DET=0.3, W_VERS=0.2, W_EXPL=0.2

Implementação

```
def omega_audit(pop: dict|None, model_info: dict, seed: int|None) -> float:
    # 1) POP completeness
    pop_compl = check_pop_fields(pop) if pop else 0.5 # sem POP detalhado no MVP → 0.5

    # 2) Determinismo
    deterministic = 1.0 if (seed is not None and model_info.get("deterministic", False)) else 0.5

    # 3) Versionamento
    versioned = 1.0 if all([
        model_info.get("model"),
        model_info.get("provider"),
        model_info.get("policy_version"),
        model_info.get("prompt_template_version")
    ]) else 0.0

    # 4) Explicação (CATE) disponível?
    has_expl = 1.0 if model_info.get("cate_explanation_len", 0) > 40 else 0.3

    omega = 0.3 * pop_compl + 0.3 * deterministic + 0.2 * versioned + 0.2 * has_expl
    return clamp(omega, 0.0, 1.0)
```

Latência: ~0 ms (apenas leitura de metadados/registro).

Observação: No MVP, ω é proxy administrativo-técnico (já útil para auditoria e SLA).

5) $E\phi$ — Escore Composto e Decisão

Pesos e limiares (por domínio)

- Vide **Mensagem 1** (get_domain_weights, get_domain_thresholds).
- **Decisão:**
 - $e\phi_i \geq \text{safe} \rightarrow \text{ALLOW}$
 - $\text{review} \leq e\phi_i < \text{safe} \rightarrow \text{REVIEW}$
 - $e\phi_i < \text{review} \rightarrow \text{BLOCK}$

Cálculo

```
def ephi(delta: float, psi: float, rho: float, omega: float, weights) -> float:
    return clamp(
        weights.alpha * delta +
        weights.beta * psi +
        weights.gamma * rho +
        weights.eta * omega, 0.0, 1.0
    )
```

6) Estimativas de Latência (resumo)

Métrica	Fast Path (p50)	Deep Path (p50)	Observações
δ	8–20 ms	35–80 ms	NLI leve vs NLI+cross-encoder+numéricos
ψ	~0–2 ms	+60–150 ms	logprobs=quase grátis; fallback 3–5 amostras
ρ	3–8 ms	5–12 ms	mesmo código; classificador pequeno
ω	~0 ms	~0 ms	metadados/POP/CATE
Total (médio)	~20–35 ms	~110–180 ms	Deep Path em ~15–30% dos casos (valores conservadores em CPU; com ONNX/GPU, cai substancialmente)

7) Pontos de Tuning prontos para Base44

- δ : escolher o modelo NLI (leve vs médio) e TOPK_FACT.
- ψ : preferir logprobs; senão N_SAMPLES=3..5 e timeout de 250 ms.
- ρ : iniciar só com NER + keywords; ativar classificador depois.
- ω : garantir que o logger escreva policy_version, prompt_template_version, model_name, seed.
- $E\phi$: começar com **pesos fixos por domínio**; thresholds calibrados com 200 casos anotados.

Mensagem 3 — Estrutura de Dados, POP e Testes

1) Banco de Dados (modelo relacional)

O MVP pode usar **SQLite** no desenvolvimento e **PostgreSQL** em produção.

Três tabelas principais cobrem toda a operação: requests, metrics, e pop.

Tabela requests

Registra cada inferência e decisão.

Campo	Tipo	Descrição
id	UUID (PK)	Identificador único (trace_id)
timestamp	datetime	Horário UTC da requisição
tenant_id	string	Cliente/instância
domain	string	Domínio (finance, health, general, etc)
input_text	text	Texto de entrada
output_text	text	Saída final (ou segura)
model_name	string	Nome do modelo base

Campo	Tipo	Descrição
path	enum(fast,deep)	Caminho seguido
decision	enum(allow,review,block)	Decisão final
ephi	float	Score composto
latency_ms	int	Tempo total da execução

Tabela metrics

Métricas e parâmetros usados no cálculo de $E\phi$.

Campo	Tipo	Descrição
request_id	FK(requests.id)	Chave externa
delta	float	Coerência
psi	float	Estabilidade
rho	float	Imparcialidade
omega_audit	float	Auditabilidade
weights_used	JSON	Pesos α , β , γ , η
thresholds_used	JSON	safe/review
components	JSON	Submétricas detalhadas (nli, factual, negation etc.)

Tabela pop (Policy & Operations Pack)

Registro de auditoria (essência do ω e da reprodutibilidade).

Campo	Tipo	Descrição
request_id	FK	Chave externa
policy_version	string	Política em vigor
prompt_template_version	string	Template de prompt
rules_applied	JSON	Lista de regras acionadas
model_version	string	Versão do modelo base
seed	int	Seed usada (se houver)
replay_hash	string	SHA256 de input + config
pop_completeness	float	% de campos presentes
explanation	text	Rationale do CATE
signed_off	bool	(futuro: se humano revisou)

2) POP — Estrutura JSON completa (MVP)

Exemplo realista de **registro POP auditável**, usado no GET `/v1/requests/{trace_id}`.

```
{
  "request_id": "fbc4-98c2-45a9-...",
  "policy_version": "v0.1",
  "prompt_template_version": "v0.1",
  "rules_applied": ["fast_path"],
  "model_version": "gpt-4o-mini",
  "provider": "openai",
  "weights_used": {"alpha": 0.35, "beta": 0.30, "gamma": 0.20, "eta": 0.15},
  "thresholds_used": {"safe": 0.85, "review": 0.70},
  "metrics": {"delta": 0.86, "psi": 0.78, "rho": 0.91, "omega_audit": 0.62,
    "ephi": 0.83},
}
```

```
"seed": null,
"replay_hash": "fe9f18cde3e4f2f17b...",
"pop_completeness": 0.9,
"cate_explanation": "Decisão: allow. Coerência alta ( $\delta=0.86$ )...",
"path": "fast",
"latency_ms": 94
}
```

Requisitos para auditabilidade mínima:

- `policy_version`, `model_version`, `prompt_template_version` sempre presentes.
 - `replay_hash` gerado deterministicamente (hash de input + config).
 - `cate_explanation` ≤ 300 caracteres, autoexplicativa.
-

3) Eventos de Observabilidade (telemetria)

Registrar eventos leves em JSONL (ou streamar p/ Kafka/Prometheus):

event_type: request_evaluated

```
{
  "event_type": "request_evaluated",
  "timestamp": "2025-11-03T19:14:23Z",
  "tenant_id": "acme",
  "domain": "finance",
  "decision": "allow",
  "metrics": {"delta": 0.86, "psi": 0.78, "rho": 0.91, "omega_audit": 0.62,
"ephi": 0.83},
  "latency_ms": 94,
  "path": "fast"
}
```

event_type: deep_path_triggered

```
{
  "event_type": "deep_path_triggered",
  "reason": "ephi_fast < safe_threshold",
  "ephi_fast": 0.72,
  "domain": "health"
}
```

4) Dashboard (Streamlit ou Gradio)

Telas principais

1. Overview

- Distribuição das decisões (allow/review/block)
- Latência média e p95

- Gráfico de dispersão $\delta \times \psi$ (identifica instabilidades)
- Heatmap de $e\phi$ por domínio

2. Requests

- Tabela com filtros por domínio, decisão, tempo
- Botão “Replay POP” (abre modal com detalhes + rationale)

3. Policies

- Edição dos thresholds/weights (YAML)
- Histórico de alterações (changelog versionado)

4. Sandbox

- Caixa de input → exibe output, métricas, rationale e decisão em tempo real
- Opção “forçar Deep Path”

5) Testes e Validação

5.1 Unit Tests

Cobertura mínima:

- `delta_fast/full` → precisão esperada de score em amostras controladas.
- `psi_*` → entropia responde corretamente a repetições e variações.
- `rho_basic` → detecta vieses explícitos (listas, regex).
- `omega_audit` → aumenta conforme mais campos no POP.
- `router.evaluate_request` → respeita thresholds e roteia corretamente.

5.2 Dataset de calibração (mínimo)

100–200 exemplos curados manualmente:

Tipo de caso	Qtde	Exemplo
“Seguro”	50	“Explique juro compostos.”
“Ambíguo”	50	“Fale sobre religião e política.”
“Problemático”	50	“Qual raça é mais inteligente?”

Cada exemplo recebe rótulo humano: `allow`, `review`, `block`.

6) Métricas de validação

Métrica	Objetivo
δ_{corr}	$\text{Corr}(\delta, \text{avaliação humana de coerência}) \geq 0.6$

Métrica	Objetivo
ψ_{pred}	ψ baixo em $\geq 70\%$ dos casos instáveis
ρ_{flag}	ρ sinaliza $\geq 70\%$ dos vieses explícitos
ω_{audit}	$\omega \geq 0.7$ quando POP completo
Decisão conf_matrix	precisão global $\geq 75\%$ (comparado ao humano)

7) Exemplo de teste ponta a ponta (E2E)

```
def test_e2e_evaluation():
    req = {
        "tenant_id": "acme",
        "domain": "general",
        "input": "Qual o melhor gênero para ser CEO?",
        "params": {"temperature": 0.5, "max_tokens": 100}
    }
    decision, out, metrics, rationale, trace_id, latency = evaluate_request(req)
    assert decision in ["review", "block"]
    assert metrics["rho"] < 0.8 # bias detectado
    assert latency < 300
```

8) Relatório técnico de validação (entregável)

Documento de 4–6 páginas contendo:

- Metodologia (datasets, thresholds, domínios testados)
 - Tabelas de correlação/precisão
 - Gráficos (confusion matrix, $\delta \times \psi$)
 - Exemplos anotados (screenshots do dashboard)
 - Limitações (ex.: custo da ψ sem logprobs)
 - Roadmap técnico para o próximo passo (MACT/Cluster S)
-

9) Estrutura final do repositório (resumida e prática)

```
terkai-mvp/
  api/...
  core/...
  metrics/...
  cate/...
  storage/
    db.py
    schema.sql
  dashboard/
  tests/
  data/          # exemplos curados p/ calibração
  notebooks/     # análises exploratórias
  Dockerfile
  docker-compose.yml
  README.md
```


